

Employee Management System - SQL Exercises

Database Schema

The Employee Management System database schema consists of the following tables:

1. Departments

Column	Data Type	Description
DepartmentID	INT (PK)	Unique department ID
DepartmentName	VARCHAR(100)	Name of the department

2. Employees

Column	Data Type	Description
EmployeeID	INT (PK)	Unique employee ID
FirstName	VARCHAR(50)	Employee's first name
LastName	VARCHAR(50)	Employee's last name
DepartmentID	INT (FK)	Linked to Departments
Salary	DECIMAL(10,2)	Monthly salary
JoinDate	DATE	Date of joining

Sample Data

Sample data for testing:

Departments:

DepartmentID	DepartmentName
1	HR
2	IT
3	Finance

Employees:

EmployeeID	FirstName	LastName	DepartmentID	Salary	JoinDate
1	John	Doe	1	5000.00	2020-01-15
2	Jane	Smith	2	6000.00	2019-03-22
3	Bob	Johnson	3	5500.00	2021-07-01

Exercises

Exercise 1: Create a Scalar Function

Goal: Create a scalar function to calculate the annual salary of an employee. Steps:

Define a scalar function named `fn\_CalculateAnnualSalary`.

```
CREATE FUNCTION fn_CalculateAnnualSalary
```

```
(  
    @Salary DECIMAL(10,2)  
)
```

```
RETURNS DECIMAL(10,2)
```

```
AS
```

```
BEGIN
```

```
    RETURN @Salary * 12;
```

```
END;
```

The function should take `Salary` as input and return `Salary \* 12`.

Test the function by selecting the annual salary for each employee.

```
SELECT
```

```
    EmployeeID,
```

```
    FirstName,
```

```
    Salary,
```

```
    dbo.fn_CalculateAnnualSalary(Salary) AS AnnualSalary
```

```
FROM Employees;
```

Exercise 2: Create a Table-Valued Function

Goal: Create a table-valued function to return employees in a specific department. Steps:

Define a table-valued function named `fn\_GetEmployeesByDepartment`.

```
CREATE FUNCTION fn_GetEmployeesByDepartment
```

```
(  
    @DepartmentID INT
```

```
)
```

```
RETURNS TABLE
```

```
AS
```

```
RETURN
```

```
(
```

```
    SELECT *
```

```
    FROM Employees
```

```
    WHERE DepartmentID = @DepartmentID
```

```
);
```

The function should take `DepartmentID` as input and return a table with employee details.

Test the function by selecting employees from the IT department.

```
SELECT *
```

```
FROM dbo.fn_GetEmployeesByDepartment(2);
```

Exercise 3: Create a User-Defined Function

Goal: Create a user-defined function to calculate the bonus for an employee. Steps:

Define a user-defined function named `fn\_CalculateBonus`.

```
CREATE FUNCTION fn_CalculateBonus
```

```
(
```

```
    @Salary DECIMAL(10,2)
```

```
)
```

```
RETURNS DECIMAL(10,2)
```

```
AS
```

```
BEGIN
```

```
    RETURN @Salary * 0.10;
```

```
END;
```

The function should take `Salary` as input and return `Salary \* 0.10`.

Test the function by selecting the bonus for each employee.

```
SELECT
    EmployeeID,
    FirstName,
    Salary,
    dbo.fn_CalculateBonus(Salary) AS Bonus
FROM Employees;
```

Exercise 4: Modify a User-Defined Function

Goal: Modify the `fn\_CalculateBonus` function to return `Salary \* 0.15`. Steps:

Alter the `fn\_CalculateBonus` function to return `Salary \* 0.15`.

```
ALTER FUNCTION fn_CalculateBonus
```

```
(
    @Salary DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
```

```
AS
```

```
BEGIN
```

```
    RETURN @Salary * 0.15;
```

```
END;
```

Test the modified function by selecting the bonus for each employee.

```
SELECT
    EmployeeID,
    FirstName,
    Salary,
    dbo.fn_CalculateBonus(Salary) AS Bonus
FROM Employees;
```

Exercise 5: Delete a User-Defined Function

Goal: Delete the `fn\_CalculateBonus` function. Steps:

Drop the `fn\_CalculateBonus` function.

```
DROP FUNCTION fn_CalculateBonus;
```

2. Verify that the function has been deleted.

```
SELECT *
FROM sys.objects
WHERE name = 'fn_CalculateBonus';
```

Exercise 6: Execute a User-Defined Function

Goal: Execute the `fn\_CalculateAnnualSalary` function. Steps:

Use the `fn\_CalculateAnnualSalary` function to calculate the annual salary for each employee.

Verify the results.

```
SELECT
    EmployeeID,
    FirstName,
    Salary,
    dbo.fn_CalculateAnnualSalary(Salary) AS AnnualSalary
FROM Employees;
```

Exercise 7: Return Data from a Scalar Function

Goal: Return the annual salary for a specific employee using `fn\_CalculateAnnualSalary`.

Steps:

Execute the `fn\_CalculateAnnualSalary` function for an employee with `EmployeeID = 1`.
Verify the result.

```
SELECT
    EmployeeID,
    FirstName,
    Salary,
    dbo.fn_CalculateAnnualSalary(Salary) AS AnnualSalary
FROM Employees
WHERE EmployeeID = 1;
```

Exercise 8: Return Data from a Table-Valued Function

Goal: Return employees from the Finance department using

`fn\_GetEmployeesByDepartment`. Steps:

Execute the `fn\_GetEmployeesByDepartment` function for `DepartmentID = 3`.

Verify the results.

```
SELECT *
FROM dbo.fn_GetEmployeesByDepartment(3);
```

Exercise 9: Create a Nested User-Defined Function

Goal: Create a nested user-defined function to calculate the total compensation for an employee.

Steps:

Define a user-defined function named `fn\_CalculateTotalCompensation`.

```
CREATE FUNCTION fn_CalculateBonus
```

```
(
    @Salary DECIMAL(10,2)
)
```

```
RETURNS DECIMAL(10,2)
```

```
AS
```

```
BEGIN
```

```
    RETURN @Salary * 0.10;
```

```
END;
```

The function should use `fn\_CalculateAnnualSalary` and `fn\_CalculateBonus` to calculate total compensation.

```
CREATE FUNCTION fn_CalculateTotalCompensation
```

```
(
    @Salary DECIMAL(10,2)
)
```

```
RETURNS DECIMAL(10,2)
```

```
AS
```

```
BEGIN
```

```
    RETURN dbo.fn_CalculateAnnualSalary(@Salary)
        + dbo.fn_CalculateBonus(@Salary);
```

```
END;
```

Test the function by selecting the total compensation for each employee.

```
SELECT
    EmployeeID,
    FirstName,
    Salary,
    dbo.fn_CalculateTotalCompensation(Salary) AS TotalCompensation
FROM Employees;
```

Exercise 10: Modify a Nested User-Defined Function

Goal: Modify the `fn\_CalculateTotalCompensation` function to include a new bonus calculation.

```
ALTER FUNCTION fn_CalculateBonus
(
    @Salary DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
AS
BEGIN
    RETURN @Salary * 0.15;
```